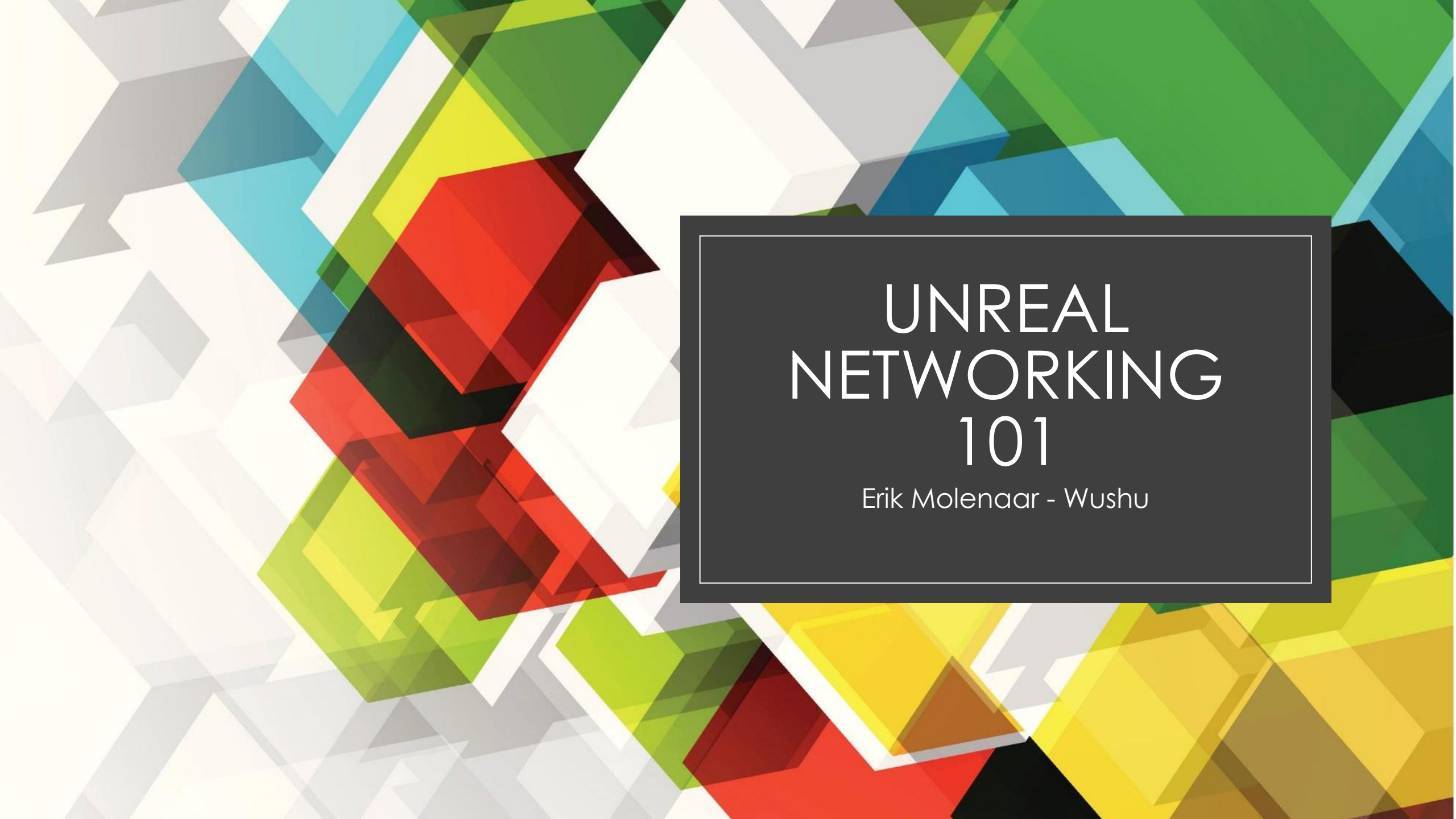
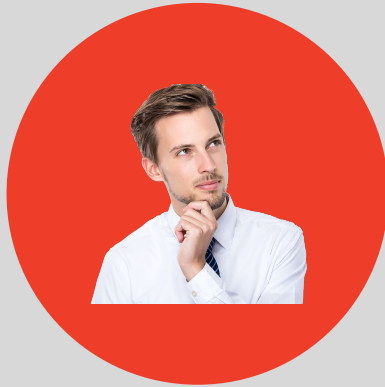




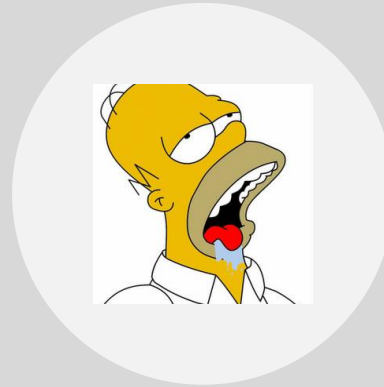
# UNREAL NETWORKING 101

Erik Molenaar - Wushu

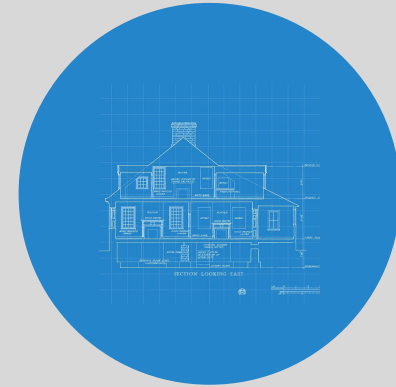
# What is this Presentation?



I WILL ASSUME EVERYONE HAS A  
BASIC UNDERSTANDING OF  
UNREAL



I WILL ASSUME NO PRIOR  
UNREAL **MULTIPLAYER**  
KNOWLEDGE



I WILL DISCUSS HOW THIS APPLIES  
TO BLUEPRINTS AS WELL AS  
CODE

Networking and multiplayer are things we are likely to run into, if you haven't already, but it can be tricky to grasp.

I built a personal project to get the hang of it.

I aim to discuss the primary topics and create a level of understanding from the ground up.



# Unreal Networking in a nutshell

- Built-in: "It just works" ™
  - Developers don't need to deal with setting up sockets, connections, serialisation, etc.
- Works through network calls and replication of data
- Server-authoritative
  - If the client wants to do something, they tell the server
  - The server makes the final call
    - Never trust the client
- Contains concepts of ownership and authority to allow or block certain actions
- Resources
  - The one-stop shop to get started: Cedric Neukirchen's Multiplayer Network Compendium  
<https://cedric-neukirchen.net/docs/category/multiplayer-network-compendium/>
    - Does gloss over some things that I wanted to go into a little more detail about, but it's a very good place to start with links to more
  - Unreal Documentation (basics, tutorials, debugging, tips)  
<https://docs.unrealengine.com/5.2/en-US/networking-and-multiplayer-in-unreal-engine/>
  - Alex Forsythe: [Multiplayer in Unreal Engine: How to Understand Network Replication \(youtube.com\)](https://www.youtube.com/watch?v=...)
  - WizardCell Unreal Multiplayer tips and tricks: [Unreal Engine Multiplayer Tips and Tricks - WizardCell](https://www.wizardcell.com/unreal-engine-multiplayer-tips-and-tricks/)



# Different instances of the game

- Unreal works with different server structures for different use-cases
- The **UGameEngine** makes a **UGameInstance** and browses to a URL to load a level, creating a **UWorld**
  - Standalone: MyMap
  - Dedicated Server, console application with no local player: MyMap
  - Listen Server (server is both client and server): MyMap?Listen
  - Client, non-server instance, browse to the server's URL: 127.0.0.1
- **ENetMode**
  - **GetNetMode()** can be called from any actor, which calls it on the **UWorld** that it is in
  - Check for server with **GetNetMode() < ENetMode::NM\_Client**
  - In Blueprint, there is no **GetNetMode**, but you can use *IsServer* or *IsDedicatedServer*

```
/**
 * The network mode the game is currently running.
 * @see https://docs.unrealengine.com/latest/INT/Gameplay/Networking/Replication/
 */
enum ENetMode
{
    /** Standalone: a game without networking, with one or more local players. Still
     * considered a server because it has all server functionality. */
    NM_Standalone,

    /** Dedicated server: server with no local players. */
    NM_DedicatedServer,

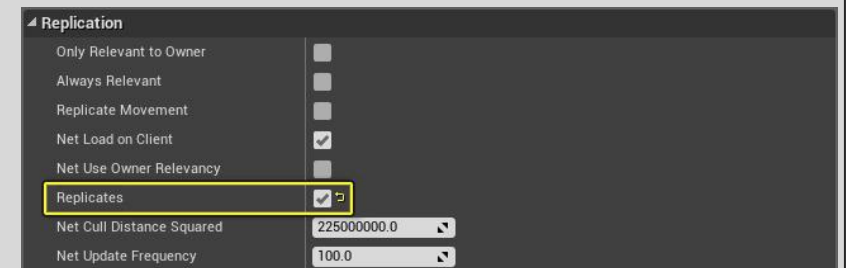
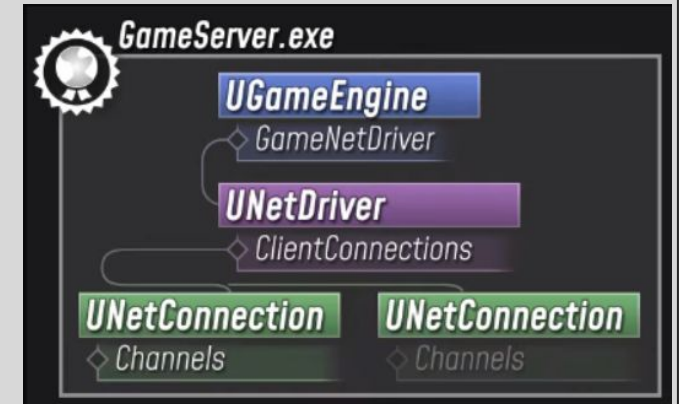
    /** Listen server: a server that also has a local player who is hosting the game,
     * available to other players on the network. */
    NM_ListenServer,

    /**
     * Network client: client connected to a remote server.
     * Note that every mode less than this value is a kind of server, so checking NetMode
     * < NM_Client is always some variety of server.
     */
    NM_Client,

    NM_MAX,
};
```

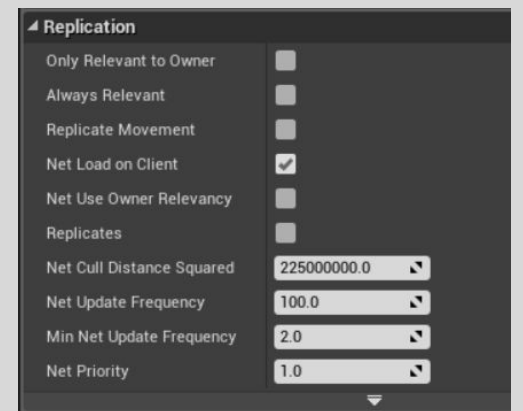
# From the start: UNetDriver

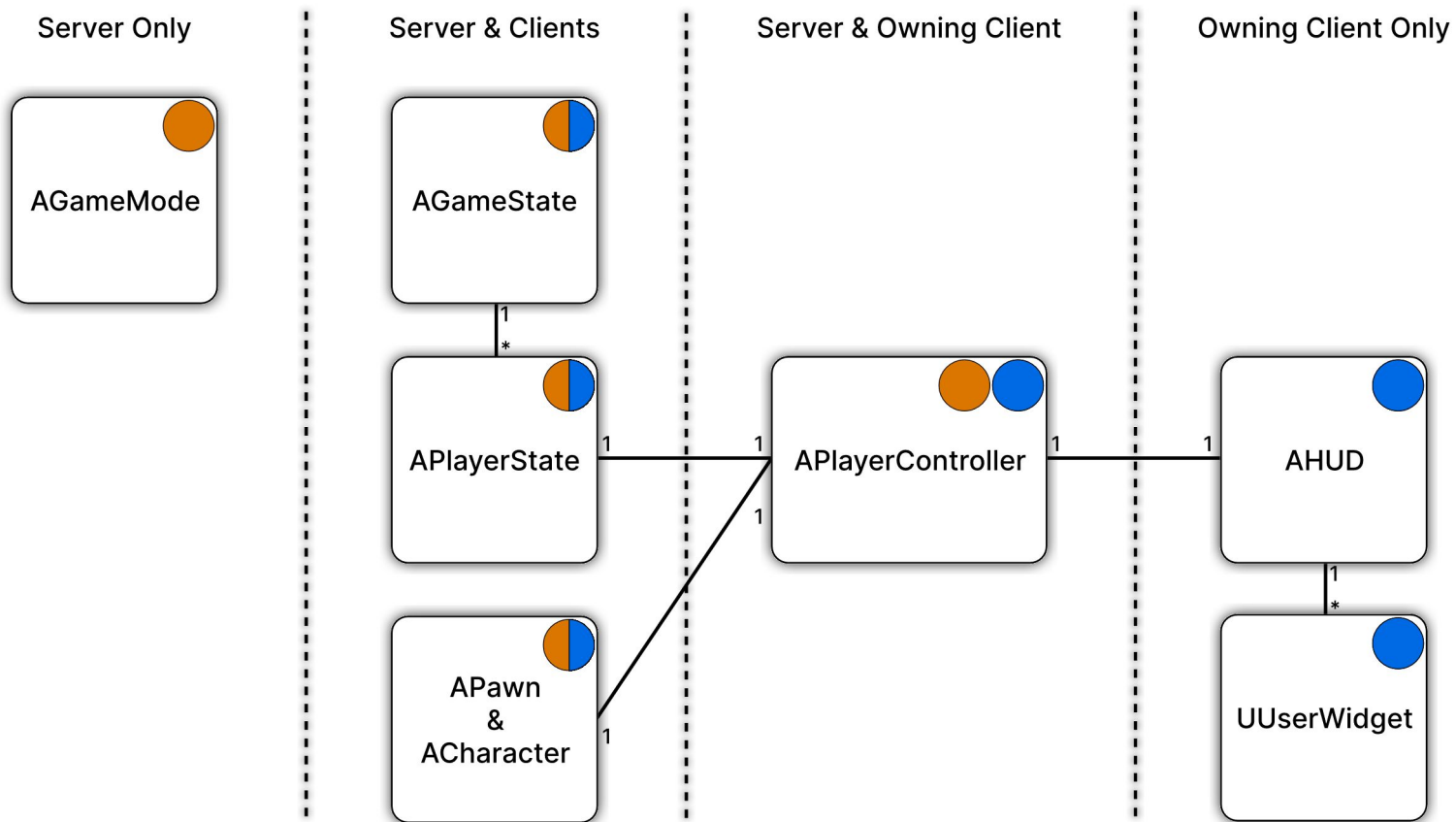
- The **UGameEngine** creates a **UNetDriver**
- The **UNetDriver** is what drives the connections, listening for or sending messages
  - The **UNetDriver** contains a list of established **UNetConnections**
- A client sends a connection request, which will lead the server to open a **UNetConnection**
  - The server has one **NetConnection** for each connected client
- An established **UNetConnection** contains a list of **UChannels**
  - **UControlChannel**, for handshakes, non-gameplay functionality, object loading, etc.
  - **UVoiceChannel**, for voice
  - And several **UActorChannels**
- There is one **UActorChannel** to communicate the data for each actor that requires replication
  - A **UActorChannel** is opened when an Actor:
    - Wants to replicate (**bReplicates = true;**)
      - Usually set in constructor, but can also be set at runtime
    - And is “relevant” to the client (e.g. nearby)



# Replication

- Core in Unreal multiplayer is replication: having a replica on the client in sync with the server's original
- Basis is **AActors**
  - The lifetime of the Actor is kept in sync (if "relevant")
- Through **AActors**, also **AActorComponents** (**SetIsReplicated(true);**) and properties
- The Actor still dictates role, priority, relevancy, culling, etc
  - Relevancy and culling: An **AActor** (with its **AActorComponents**/properties) is only replicated when relevant
  - Relevancy is the result of **bool IsNetRelevantFor(Player)**, which can be overridden to implement bespoke rules
    - Default is a check **distanceSquared < NetCullDistanceSquared**, with early out checks for:
      - **bAlwaysRelevant || IsOwnedBy(Player)** (see Ownership)
        - Examples that are always relevant include **AGameStateBase** and **APlayerStates**
      - **bNetUseOwnerRelevancy**, asking higher up the ownership chain
      - **bOnlyRelevantToOwner** (such as **APlayerControllers**)
      - Attached as child actor or to a **SkeletalMesh** (the parent will handle it)
      - **bHidden** && no collision (in which case it is not relevant)



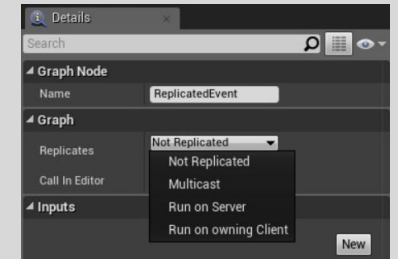
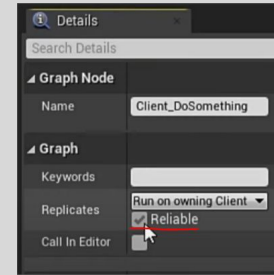


# Common Classes

- These are the common, core Unreal classes and where they live in multiplayer
- This is the foundation for what action and logic to perform where

# Remote Procedure Calls (RPCs)

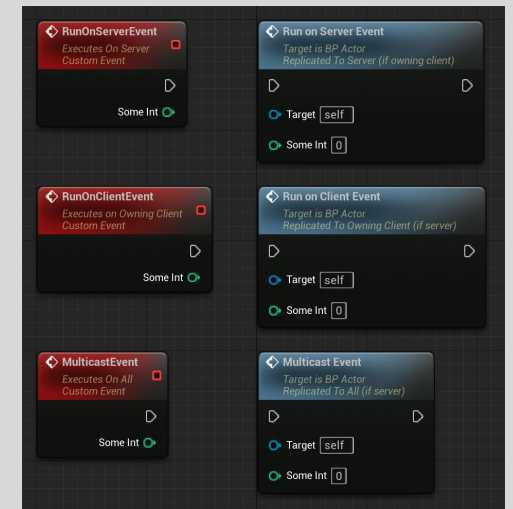
- Send a function call to other instances of the game “instantly”
  - One of the two primary ways of sharing data between instances
  - The only way to send data from the client to the server!
- Three types (usually indicated with prefixes)
  - Multicast RPC: Intended to be called from the Server, to Clients and Server
  - Server RPC: Intended to be called from a Client, to the Server
  - Client RPC: Intended to be called from the Server, to a specific client (the owner of the object the function is called on)
- Desired RPC reliability can be indicated too, unreliable by default
  - `UFUNCTION(Client, Reliable)`: Reliable RPCs guaranteed to arrive and, within a single Actor, guaranteed to arrive in order of being called, but comes with a potentially higher cost
  - `UFUNCTION(Client, Unreliable)`: Unreliable, cheaper, but not guaranteed
- In Blueprint, the implementation is simply the defined function with the intended settings
- In C++ you need to suffix the function name with `_Implementation`
- We can add a server-side validation function using `WithValidation` in the `UFUNCTION` declaration
  - This can be implemented using the `_Validate` suffix on the function implementation



```
UFUNCTION(NetMulticast)
void Multicast_DoSomething();

UFUNCTION(Server)
void Server_DoSomething();

UFUNCTION(Client)
void Client_DoSomething();
```



```
// .h
UFUNCTION(NetMulticast, Reliable, WithValidation)
void Server_DoSomething();

// .cpp
void AMyActor::Server_DoSomething_Validate() { /* validation goes here */ }

void AMyActor::Server_DoSomething_Implementation() { /* implementation goes here */ }
```

# Remote Procedure Calls (RPCs)

- There are a few requirements that need to be met for RPCs to be completely functional:
  1. They must be called on Actors or a replicated SubObject (e.g. a component)
  2. The Actor (and component) must be replicated
  3. If the RPC is being called by the server to be executed on a client (so a client RPC), only the client who **owns** that Actor will execute the function
  4. If the RPC is being called by a client to be executed on the server (so a Server RPC, the client must **own** the Actor that the RPC is being called on
  5. Multicast RPCs are an exception:
    - If they are called by the server, the server will execute them locally, as well as execute them on all currently connected clients, which have an instance of that Actor that is **relevant**
    - If they are called from clients, a Multicast will only execute locally, and will not execute on the server or other clients
    - For now, we have a simple throttling mechanism for Multicast events: A Multicast function will not replicate more than twice in a given Actor's network update period.  
Long term, Epic expects to improve on this.

# Ownership

- Affects Actor relevancy, replication, RPCs and property replication
  - Usually, if RPCs act in unexpected ways, look at ownership
- Ownership describes who 'owns' an **AActor**
  - For the server: Decides where RPCs are executed
  - For the clients: Decides who can call RPCs on this Actor
- Ownership starts at the **UNetConnection**, which has a connection to the relevant player's **APlayerController**
  - That means that PlayerController is owned by the Client that Connection is for
  - Anything owned by the PlayerController is also owned by that client
    - This includes possessing a Pawn (but only *while* possessing)
- Can be checked through **IsOwnedBy(AActor\*)** in code, no real alternative in Blueprint
  - In Blueprint you can try getting the related controller and checking it against the local controller

## RPC invoked from the Server

| Actor Ownership    | Not Replicated | NetMulticast                   | Server         | Client                        |
|--------------------|----------------|--------------------------------|----------------|-------------------------------|
| Client-owned Actor | Runs on Server | Runs on Server and all Clients | Runs on Server | Runs on Actor's owning Client |
| Server-owned Actor | Runs on Server | Runs on Server and all Clients | Runs on Server | Runs on Server                |
| Unowned Actor      | Runs on Server | Runs on Server and all Clients | Runs on Server | Runs on Server                |

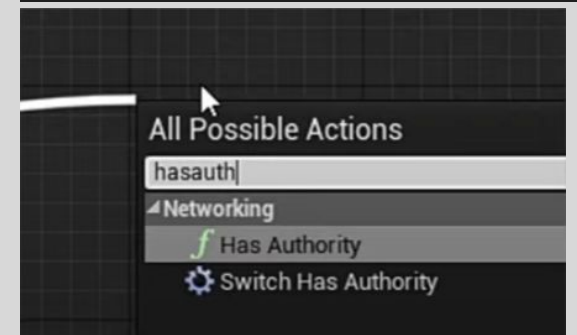
## RPC invoked from a Client

| Actor Ownership             | Not Replicated          | NetMulticast            | Server         | Client                  |
|-----------------------------|-------------------------|-------------------------|----------------|-------------------------|
| Owned by invoking Client    | Runs on invoking Client | Runs on invoking Client | Runs on Server | Runs on invoking Client |
| Owned by a different Client | Runs on invoking Client | Runs on invoking Client | Dropped        | Runs on invoking Client |
| Server-owned Actor          | Runs on invoking Client | Runs on invoking Client | Dropped        | Runs on Invoking Client |
| Unowned Actor               | Runs on invoking Client | Runs on invoking Client | Dropped        | Runs on Invoking Client |

# Roles and Authority

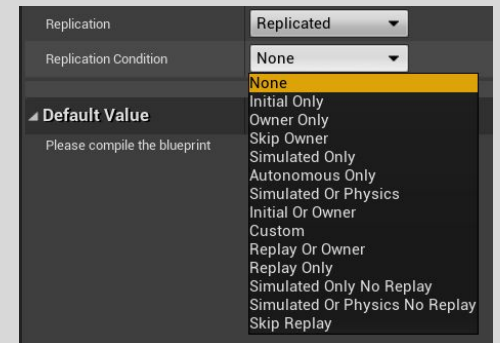
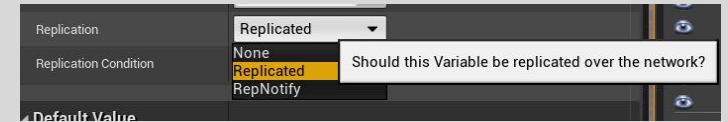
- When actors are spawned, they get a Role (**ENetRole**) assigned to them
- The Role is **ROLE\_Authority** on the system that initially created the actor
  - If an Actor needs to be replicated, it must be spawned on the server and be assigned **ROLE\_Authority** on the server
  - An actor can still be spawned on the client but will not replicate. It will still be assigned **ROLE\_Authority** on that client
- If a server-spawned Actor is replicated to a client, it will be a proxy for the original and be assigned either:
  - **ROLE\_SimulatedProxy**, for Actors controlled remotely (this is the case for most replicated Actors)
  - **ROLE\_AutonomousProxy**, for Actors that are controlled locally, for which the changes are sent to the server, and allowing for ServerRPCs to be called (most commonly PlayerController and its possessed Pawn)
- Though **GetLocalRole()** gives you the exact role, the generally more useful function is **AActor::HasAuthority()**
  - Authority means you have the final say in changing the state of the Actor
  - **HasAuthority** is **not** the same as “Am I the server?” as it depends on what Actor you call it on
  - It is also **not** the same as “Does the local player have ownership?”
    - *GetLocalRole == ROLE\_AutonomousProxy* is very close, but I feel like it will have exceptions as **SetOwner()** does not update the role
  - There is a **LocalRole** and a **RemoteRole** to define the role it has locally and the role it will get when replicated (you get the reverse when checked from the client)
- Tip: prefix a function name with e.g. **Auth** when setting up functions only to be run with authority

```
UENUM()
enum ENetRole
{
    /** No role at all. */
    ROLE_None,
    /** Locally simulated proxy of this actor. */
    ROLE_SimulatedProxy,
    /** Locally autonomous proxy of this actor. */
    ROLE_AutonomousProxy,
    /** Authoritative control over the actor. */
    ROLE_Authority,
    ROLE_MAX,
};
```



# Property Replication

- The other of the two main ways to facilitate data communications
- Whereas RPCs are “immediately”, property replication is “eventually”
  - However, a big advantage is that this includes *after* a client has newly connected
    - RPCs are in this case missed as the client wasn't there when the call was made
  - Built-in load balancing to not overload the connection with messages, meaning the update may be delayed if needed
- Mark property with `UPROPERTY(Replicated)` or set to Replicated in Blueprint
- `ReplicatedUsing/RepNotify` calls the indicated function (often prefixed with `OnRep_`) when the corresponding property is replicated
  - Note that Blueprint replication callback functions are automatically called when changing the value even on the server, but will need to be explicitly called in C++ to run the logic on the server too
- `GetLifetimeReplicatedProps` needs to be implemented, notifying the system of what properties need to be replicated
  - This is done with the `DOREPLIFETIME([Actor Class], [Property])` macro
  - There are variations to this, the most common being `DOREPLIFETIME_CONDITION`, allowing the addition of a replication condition



```
UPROPERTY(Replicated)
bool bMyValue;

UPROPERTY(ReplicatedUsing = OnRep_MyFunctionReplicatedValue)
bool bMyFunctionReplicatedValue;

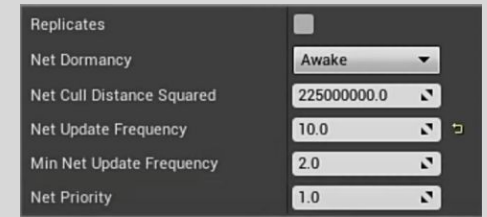
UFUNCTION() // called when bMyFunctionReplicatedValue is replicated
void OnRep_MyFunctionReplicatedValue(bool bOldValue);

// This function is needed to make replication work; also needs include for UnrealNetwork.h
virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const override
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    DOREPLIFETIME(AMyActor, bMyValue);
    DOREPLIFETIME_CONDITION(AMyActor, bMyFunctionReplicatedValue, COND_OwnerOnly);
}
```

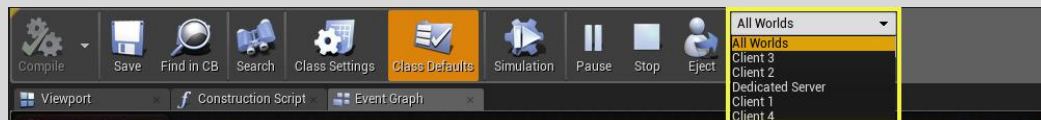
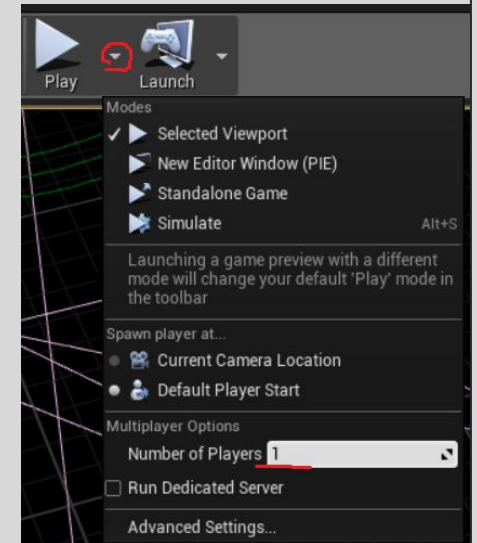
# Update Frequency, Priority, Bandwidth & NetDormancy

- The replication system's **UNetDriver** has a finite amount of bandwidth at any given time
  - To deal with that it has a built-in load balancing system
- The **UNetDriver** will send updates for Actors based on their priority until the bandwidth is used up or all updates are sent
- This Actor network priority by default (see `AActor::GetNetPriority(***)`) is affected by :
  - Distance to the player (closer is higher priority)
  - Time not updated to clients
  - Net Priority, functions as a multiplier
- `NetUpdateFrequency` says how often per second an Actor will be considered for replication
  - There is a real-world limit given internet connections and latency
- `NetDormancy` allows for some more fine-grained control over when an Actor updates or not
- Not every property update is sent right away; the value will reach the client *eventually*



# Multiplayer in the Editor

- We can test multiplayer behaviour by launching multiple instances of PIE
  - From the Play dropdown, increase the Number of Players
  - The first instance will be a ListenServer, all others are connecting clients, indicated in the window bar
    - You can use Shift+Tab and Control+Tab to cycle the focus between your client windows
  - By checking the Run Dedicated Server checkbox, there is no ListenServer, but a dedicated server
- Some systems, like external plugins, might not play nice with local testing
- Advanced settings allow for further customisation and maybe fixing potential issues
  - An interesting one to note here is Network Emulation, allowing emulation of poor network conditions, such as latency and packet loss
- To debug blueprints, you can filter by the world they refer to
  - This setting also means breakpoints will only trigger for that world
  - When breaking on breakpoints, a text overlay will tell what world the breakpoint is active for



# In Practice

- Actually implementing it means making impactful choices
  - Ideally you make informed choices, this is why having this basis can be quite important
- What happens where, when and why?
- Fire a gun, hit trace, effects, damage, feedback (like hit flash, damage sounds)
  - Where do you draw the lines for what happens on which instance?
  - Client controls, so they tell the server they are shooting
    - But we don't want to wait with effects, so we start things locally already for the local player
  - Multiple clients need to know, so the server multicasts
    - Controlling client receives this as well, but is already shooting, so filters this out
  - We work with a server authoritative system, so the server deals the damage and updates the clients
  - The clients get damage updates, update the values and show the feedback

# General Notes

- Networking purely through Blueprints is somewhat limited and can sometimes have potentially unintended side effects
- Beware of **GetPlayerXXX(0)** Static Functions
  - This *usually* returns the local player, or first client (including local client for ListenServer), but if another client connected first after a map switch, this may result in returning another client instead
  - Instead, follow ownership chains to get the relevant objects, or `UGameInstance::GetFirstLocalPlayerController()`
- Be aware that RPCs can be missed
  - When an Actor is not deemed relevant on the client
  - When an Actor is not yet replicated
  - When a client is not yet connected
- Don't use delays to make up for replication time; use **ReplicatedUsing/RepNotify** instead
- Be careful with overriding RPCs (in C++)
  - Since it relies on a chain of generated functions, it's easy to do wrong
  - Instead, you can make an RPC that calls an overridable function and override that instead
- Beware of replicating **Ftext**
  - *"Avoid replicating formatted FTexts when possible, each FText is sent as three FString, and they also send the entire formatting history. A simple "Bob's Home" string can replicate over 30 FString."*
- For replicating (large) TArrays of UStructs, use fast array replication using **FFastArraySerializerItem**
- New **Experimental** systems to keep an eye on: Push Model replication (UE4.25) and Iris (still Beta as of UE5.4)
  - These don't rely on polling property changes, but on actively marking them dirty when needed, requiring more work, but better performance
- To replicate many Actors, consider looking at Replication Graph (since UE4.20, used in Fortnite)

# Brief look at what lies beyond

- Efficient replication
- OnlineSubsystems
  - OnlineSubsystemNull is default
  - Sessions
- Specific component behaviours like CharacterMovementComponent
- Determinism
  - Physics and replication
- Order of actions (lag & ping)
  - Rollback
- Stat Net

“As I look into the abyss,  
the abyss looks into me.”



# References

- The one-stop shop to get started: Cedric Neukirchen's Multiplayer Network Compendium <https://cedric-neukirchen.net/docs/category/multiplayer-network-compendium/>
- Unreal Documentation (basics, tutorials, debugging, tips) <https://docs.unrealengine.com/5.2/en-US/networking-and-multiplayer-in-unreal-engine/>
- Alex Forsythe: [Multiplayer in Unreal Engine: How to Understand Network Replication \(youtube.com\)](https://www.youtube.com/watch?v=...)
- WizardCell Unreal Multiplayer tips and tricks: [Unreal Engine Multiplayer Tips and Tricks - WizardCell](https://www.wizardcell.com/unreal-engine-multiplayer-tips-and-tricks/)